Motor ECU 개발 경험 자기소개서 활용 노트

임베디드 펌웨어 · 차량 네트워크 · 검증 직무 2026년 10월 17일 작업 중심

경험을 설명하는 중심 문장

CAN 기반 Motor ECU의 제어·통신·출력 경계를 정리하고, RTOS 동시성 구조와 DBC 인터페이스를 코드·시험 도구·문서에 일관되게 반영한 경험이다.

기술스택과 실제 사용 목적

분야	사용 기술	이번 작업의 역할
MCU / 언어	STM32F446RE, C, STM32 HAL	CAN 수신·송신, 타이머 PWM, 드라이버/서보 출력
실시간 OS	FreeRTOS V10.3.1 + CMSIS-RTOS2	5 tasks, mutex 2개, queue 2개, 우선순위 상속
차량 통신	Classic CAN 500kbps, DBC	0x100 명령 / 0x110 상태, Alive·timeout 검사
물리 출력	MDD3A 2개, DC 4개, SG90 2개	20kHz DC PWM, 50Hz 서보 PWM, 전원/GND 분리
PC / 검증	Python, CANgaroo, CANable, GCC	주기 송신·자동 시퀀스, mock 회귀, 전체 ARM 빌드
환경 / 산출물	STM32CubeIDE 2.2.0, Git, Markdown	원격 develop 비교, 핀 입출력·구성도·검토 기록

기록에서 확인되는 결과

전체 ARM 빌드 통과, C 제어 회귀와 CMSIS2 task 모의 회귀 통과, 최신 Python script **15개 테스트 통과**. 공식·시험용 DBC와 펌웨어/스크립트의 조향 배치를 Byte4~5로 동기화했다.

수치의 의미: 테스트 수는 Python unittest 기준이다. 서로 다른 C 테스트를 합산하지 않았다. 1ms/10ms는 설정 주기이며 실물 최대 지연이나 성능 향상률을 측정한 값이 아니다.

01. 작업 과정과 기여 정리

오늘의 변경과 앞선 시행착오를 연결하되, 수행 주체와 검증 수준을 구분한다.

단계	진행 내용	산출물 / 의미
요구 파악	VCU 요청 → Motor ECU → DC/서보 출력 구조 확인	실측 모델과 Virtual Motor 모델 구분
기존 문제 추적	TX 증가·RX0, RX/TX 배선 착오와 PC 설정 확인	사용자 배선 수정 후 fresh 0x110/READY 로그
RTOS 전환	CMSIS v1 API에서 v2로 전환, RX/제어/출력/TX/Health 분리	ISR는 큐 전달, 공유 상태는 mutex로 보호
명세 교차 검토	원격 develop 226e88e의 아키텍처·DBC·Excel 대조	조향 Byte3~4와 Body 후보 Byte3의 중복 발견
인터페이스 수정	조향을 Byte4~5로 이동, firmware4/script1.4 적용	기존 Torque/Gear/Brake/Alive 위치 유지
반복 시험 도구	script1.5 auto 구현: 속도·좌우 조향·전후진 반복	30초 순환, 정지 구간, 통신 고장 시 자동 해제
전달·재현성	CN7/CN10 핀 번호와 전원·GND·종단 정리	하드웨어 Pin In/Out 문서 및 연결 확인표

내 역할을 작성할 때의 기준

사용자는 DC 4개·서보 2개 요구를 제시하고 실제 배선·IDE·CANgaroo 실행 로그를 공유했다. RX/TX 배선 착오를 직접 확인해 수정한 사실은 대화에 남아 있다. 코드 변경·테스트 작성·문서 작성은 AI 보조로 진행되었다.

자기소개서의 "설계했다·구현했다"는 표현은 본인이 코드를 설명하고 재현할 수 있는 범위에서 사용한다. 팀 전체 소유권, 혼자 작성한 코드량, 실물 최종 검증 완료 등을 이 기록만으로 주장하지 않는다.

주도성을 보여 주는 근거

도구를 사용했다는 사실보다 요구를 구체화한 과정, 로그를 가져와 가설을 좁힌 과정, 다른 ECU와 충돌하는 규격을 확인하고 수정 범위를 일관되게 정한 판단을 설명한다.

02. 문제해결 사례를 STAR로 묶기

사례 A는 인터페이스 정합성, 사례 B는 통신 문제의 원인 분리 역량을 보여 준다.

사례 A | 단독 시험에서 보이지 않던 DBC 비트 충돌

STAR	기록에 근거한 내용
Situation	DC·서보를 함께 제어하는 시험 확장에서는 조향각을 0x100 Byte3~4에 배치했다. 최신 Gateway Routing 문서에는 Byte3를 램프·도어·윈도 명령으로 쓰는 후보안이 있었다.
Task	다른 ECU와 통합했을 때 서로 다른 기능이 같은 바이트를 해석하지 않도록, 인터페이스 충돌을 제거해야 했다.
Action	최신 원격 커밋의 DBC·메시지 정의·Routing을 대조했다. 조향 -20도의 raw -200이 38 FF가 되어 Body 후보에서는 Hazard 등의 값으로 읽힐 수 있음을 확인했다. Byte4~5로 이동하고 DBC·C 디코더·Python 송신·테스트를 함께 바꿨다.
Result	Byte3 전체 256가지 값이 조향·기존 DC 명령에 영향을 주지 않는 회귀를 통과했다. firmware revision4와 script의 버전 검사를 맞춰 구버전 혼용을 줄였다. 팀 검토·원격 반영과 전체 ECU 통합은 별도다.

사례 B | “TX는 늘지만 RX는 0”인 상태의 진단

상황: 스크립트 송신 카운터는 증가했지만 0x110 수신이 없었다. 모터 전원과 CAN 통신의 원인을 섞어 판단하기 쉬운 상태였다.

접근: PC interface 선택, 500kbps와 Listen-only, MCU 실행 상태, 트랜시버 TX/RX 배선을 구분해 점검했다. 사용자에게서 RX/TX 연결 착오가 확인되었고, 이후 RX 증가·READY·수 ms 수준의 fresh status 로그가 관찰되었다.

배운 점: PC의 TX API 호출 성공은 상대 ECU 수신 성공이 아니다. CAN 상태 수신과 실제 PWM·모터 전력은 별도 경로이므로 단계별 관측점을 둬야 한다. 로그 하나만으로 최초 원인을 모두 확정했다고 표현하지 않는다.

03. 자기소개서에 바로 활용하는 초안

지원 직무와 글자 수에 맞춰 다듬을 재료다. 개인이 설명·재현 가능한 내용만 제출한다.

경험 중심 초안 | 인터페이스 정합성과 문제해결

초안

CAN-LIN Gateway 프로젝트에서 Motor ECU의 제어 인터페이스와 검증 흐름을 다뤘습니다. DC 모터 네 개와 서보 두 개를 연결하는 과정에서 단독 시험의 동작 여부만으로는 통합 정합성을 판단할 수 없다고 보았습니다. 최신 develop의 아키텍처, DBC와 Gateway Routing을 비교한 결과, 시험용 조향각이 사용하는 Byte3와 Body 명령 후보 영역이 겹치는 점을 확인했습니다. 조향각을 Byte4~5로 이동하고 DBC, 펌웨어 디코더, PC 송신 스크립트와 회귀 테스트를 함께 정렬했습니다. 또한 구버전 펌웨어를 식별하도록 버전 검사를 갱신했습니다. 이 경험을 통해 임베디드 개발의 완성도는 개별 기능의 구현뿐 아니라 인터페이스의 일관성과 재현 가능한 검증에서 나온다는 점을 배웠습니다.

직무 역량 초안 | 실시간 구조와 검증 범위

Motor ECU를 CMSIS-RTOS2 기반으로 정리하며 수신, 제어, 출력과 송신의 책임을 나누었습니다. ISR에서는 블로킹 없이 큐에 프레임을 전달하고, task 사이의 공유 상태는 우선순위 상속 mutex로 보호했습니다. 제어와 출력 task가 각각 명령 freshness를 확인하도록 하고, 큐 포화와 mutex 획득 실패 상황도 모의 테스트했습니다. 전체 ARM 빌드와 회귀 테스트 통과 후에도 실물 지연, bus-off 복구와 CPU 예외 시 출력 차단은 별도 검증 과제로 남겼습니다. 구현한 기능과 검증하지 못한 영역을 구분해 설명하는 개발 습관을 갖추고자 했습니다.

이력서에 넣을 수 있는 짧은 표현

- STM32 Motor ECU의 CMSIS-RTOS2 task/queue/mutex 구조 및 CAN 명령·PWM 출력 연동 정리
- 조향각과 Gateway Body 명령 후보의 DBC 비트 중복 분석, Byte4~5 이동 및 송수신 규격 동기화
- CANgaroo 자동 시퀀스·고장 중단 동작 구현 보조 및 Python 회귀 15개 통과

제출 전 개인화할 항목

지원 회사의 요구 역량과 연결되는 사례 하나를 중심으로 고른다. 본인의 팀 내 책임, 실제 보드에서 직접 확인한 동작, 팀 리뷰 결과를 추가한다. AI 보조 범위를 질문받으면 코드 초안·변경·테스트 작성에 활용한 점과 본인이 검토한 판단을 사실대로 구분한다.

04. 성과를 증거로 설명하기

완료·관찰·미확인을 섞지 않는 것이 면접에서 신뢰를 만든다.

구분	표현 가능한 사실	추가로 필요한 증거
구현 / 시험	C 제어 회귀 PASS, RTOS mock PASS, Python 15 tests PASS	테스트 명령과 로그, 실패 사례를 설명할 수 있어야 함
빌드	revision4 전체 ARM Debug 빌드 통과	실제 보드의 실행 버전과 연결 확인
실물 관찰	RX 증가·READY/fresh status 로그, 서보 회전량이 작다는 사용자 관찰	최신 auto에서 DC4/서보2 전체 동작·정지 영상/로그
모델 값	CAN RPM/TorqueApplied는 Virtual Motor 기준	실측 RPM·전류·토크라고 표현하지 않음
설계 수치	제어/출력 1ms, CAN10ms, age>30ms inhibit	실물 최대 응답시간·jitter·mutex 점유시간 미측정
미해결	CPU fault PWM 차단, bus-off 복구, 3회 확인 계수 정의	해결 전 "고장 복구 완성"이라고 작성하지 않음

추가 측정 후 채울 성과 항목

실물 timeout 검출~출력 차단 최대 지연: [측정값]ms / 시험 부하: [조건]
자동 시퀀스 연속 운전: [시간]분 / CAN 프레임 누락·fault 발생: [횟수]
task별 최소 stack 여유: [bytes] / 최대 mutex 보유: [시간]
버스 오프 및 통신 재개: [절차·소요 시간] / 서보 각도 보정 오차: [측정값]

나중에 다시 열어 볼 저장소 산출물

Core/Src/freertos.c · motor_can.c · motor_app.c · motor_watchdog.c
cangaroo/motor_control.py · tests/test_cangaroo_script.py
firmware/motor_ECU/RTOS2_ARCHITECTURE.md
firmware/motor_ECU/DEVELOP_ARCHITECTURE_REVIEW.md
network/can/MOTOR_STEERING_INTERFACE.md
docs/hardware/MOTOR_ECU_PINOUT.md

경로는 저장소 루트 freertos-can-lin-gateway 기준이다. 원격 검토 기준은 develop 커밋 226e88e076171037d9e115e9722ec3cbdc18e268. 현재 로컬 변경이 원격에 push되었다는 뜻은 아니다. 본 문서는 대화와 저장 파일을 요약한 취업 준비용 자료이며 정식 시험성적서는 아니다.